Experiment-Based Understanding of Deep-Learning Architectures: ResNet-152, Vision Transformers, CLIP, and Variational Autoencoders

Talha Masood
Student ID: 2028-10-0131
CS-6304 - Advanced Topics in Machine Learning
28100131@lums.edu.pk

Abstract

This report investigates the internal mechanisms of four deep learning architectures through targeted experiments. For ResNet-152, the aim was to examine how well pretrained convolutional features transfer to a new dataset and how residual connections affect gradient flow and feature stability across depth. For Vision Transformers, the aim was to study how self-attention builds an image representation and whether its built-in attention weights offer a more direct form of interpretability compared to CNN methods such as CAM and Grad-CAM, which need an extra calculation after the prediction is made. For CLIP, the aim was to assess zero-shot transfer across different prompting strategies and to examine the geometric relationship between its image and text embedding spaces using the Procrustes transform. For the variational autoencoder, the aim was to examine how a generative latent-variable model organizes and reconstructs data under a low-dimensional latent bottleneck.

1 Task 1: ResNet-152

1.1 Baseline Setup

1.1.1 Implementation

A pretrained ResNet-152 model (trained on ImageNet-1k weights) was used, where all backbone parameters were frozen (`requires_grad=False`). The final 1000 class classification layer was replaced with a randomly-initialized linear layer (`nn.Linear(2048,10)`) mapping the 2048-dimensional pooled feature vector to the 10 CIFAR-10 classes. CIFAR-10 images were resized to 224×224 and normalized using ImageNet channel statistics (mean and standard deviation) to match the distribution the backbone was pretrained on and prevent the occurrence of any excess loss due to feature scaling mismatches. The 50,000 image training set was split into 45,000 training and 5,000 validation examples and the official 10,000 image test set was used only for the final evaluation. Only the new linear head (20,490 parameters) was trained, using the Adam optimizer (lr = 0.001) and cross-entropy loss, for 5 epochs.

1.1.2 Findings and Discussion

The entire 58-million-parameter backbone frozen and only a trained 20,490 parameter linear head , the model reached 86.08% train, 84.64% validation, and 85.26% test accuracy after just 5 epochs. This shows that the backbone's ImageNet pretrained parameters already encode general-purpose

Preprint.

Table 1: Baseline frozen-backbone results (5 epochs, Adam, lr=0.001).

Split	Loss	Accuracy
Train (final epoch)	0.4109	86.08%
Validation (final epoch)	0.4438	84.64%
Test	0.4529	85.26%

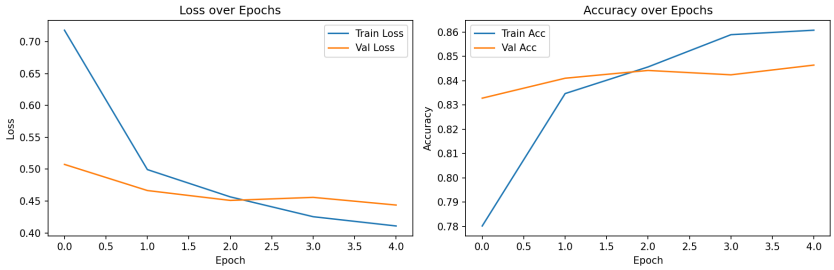


Figure 1: Training and validation loss/accuracy over 5 epochs, frozen ResNet-152 backbone.

visual features that transfer well enough to CIFAR-10 that a single linear layer, with no parameter updates to the backbone itself, is sufficient to achieve strong classification performance. Training a network with this high of a parameter count from random initialization on CIFAR-10's comparatively small 50,000-image training set would likely warrant substantially more training time and careful regularization to reach comparable generalization performance, given the far lower sample-size to parameter-count ratio than the approximately 1.2 million images ResNet-152 was originally pretrained on. As such, overfitting risk with this setup is a genuine outcome under bias-variance understanding, though phenomena such as double descent complicate any simple claim that a low sample-size to parameter-count ratio guarantees overfitting on its own. Validation accuracy climbed from 83.28% to 84.64% over the 5 epochs, with a small dip at epoch 4 (84.24%, down from 84.42% at epoch 3) before recovering. The overall upward trend suggests accuracy had not yet fully plateaued, and additional epochs may yield further gains, though the diminishing size of epoch-to-epoch improvements (roughly +5 points from epoch 1 to 2, but under +1 point from epoch 3 to 5) suggests the linear head was already approaching its practical ceiling given the frozen backbone. One aspect that may have affected results but was built into the task was CIFAR 10's resizing. CIFAR-10's native 32×32 resolution was increased to 224×224 to match the backbone's expected input size. This increase does not add genuine image detail, the additional pixels are interpolated, not newly observed detail, so the effective visual information available to the model remains bounded by the original low resolution. This is a plausible contributing factor to accuracy not being of a higher magnitude, independent of the backbone's feature vector quality.

Regarding transferability specifically: the fact that freezing nearly the entire network and training only a linear classifier still achieves over 85% test accuracy is strong evidence that ResNet-152's learned representations are not narrowly specialized to ImageNet's original 1000-class task, but capture broadly reusable visual structure. Although, this subtask alone cannot determine which specific features (edges, textures, or higher-level patterns) are responsible for this transferability, that question is touched upon in Subtask 3's feature-hierarchy analysis

1.2 Residual Connections in Practice

1.2.1 Implementation

Skip connections were removed from the first bottleneck block of layer1, layer2, and layer4 (early, middle, and late network depth) by creating a subclass `BottleneckNoSkip` that inherits from the `Bottleneck` module of torchvision and overriding the `forward` method of the parent class to omit the `out += identity` addition, while leaving all convolutional and batch-normalization weights untouched, implementing a class reassignment for the existing block instances of the pretrained model. Two experiments were run:
(i) a one-batch, one-pass gradient-magnitude test, with the backbone temporarily unfrozen, comparing

the gradient norm at `layer1[0].conv1.weight` between the baseline and modified architectures. (ii) a full retraining of the classification head (identical setup to the baseline: frozen backbone, Adam, lr = 0.001, 5 epochs) on the modified architecture, for direct comparison with the baseline.

1.2.2 Findings and Discussion

Table 2: Skip-connection ablation: gradient diagnostic and retrained-head results.

Metric	Value
Gradient norm, intact model (layer1[0].conv1)	4.5359
Gradient norm, modified model (layer1[0].conv1)	3.9394
Gradient ratio (modified / intact)	0.8685
Modified model – Test Loss	1.6135
Modified model – Test Accuracy	43.79%

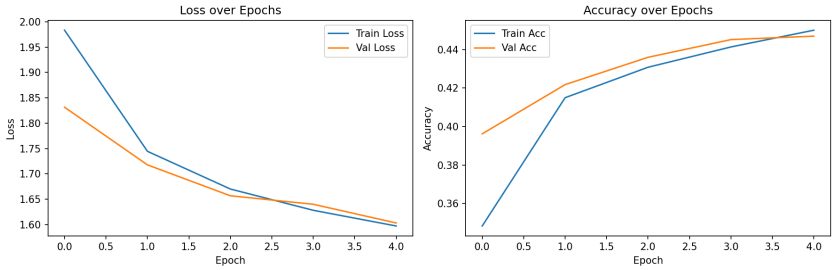


Figure 2: Training and validation curves, skip-connection-modified ResNet-152 (blocks layer1[0], layer2[0], layer4[0] modified).

Skip connections address the vanishing-gradient problem in deep networks such as ResNet by adding the block's input directly to its pre-activation output. For a residual block computing output $= F(x) + x$, the derivative with respect to the input is $\partial \text{output}/\partial x = \partial F(x)/\partial x + 1$. This additive $+1$ term guarantees a gradient path that is not diminished by how diminutive the derivative of F itself might be, which is what prevents the multiplicative shrinkage that causes vanishing gradients when many such blocks are chained together. In this experiment, the measured gradient-flow effect was present but not highly significant in magnitude. Removing skip connections from three of ResNet-152's fifty bottleneck blocks (`layer1[0]`, `layer2[0]`, `layer4[0]`) reduced the gradient norm at `layer1[0].conv1.weight` from 4.536 in the intact model to 3.939 in the modified model, a ratio of 0.868, so only a 13% reduction. This was expected as the remaining 47 intact blocks each still contribute their own $+1$ term to the backward chain, so they provide plenty of alternative gradient pathways (producing less overall reduction in the magnitude of the gradient signal) that prevents a strong vanishing gradient effect. Despite this small change in gradient flow, test accuracy dropped from 85.26% in the baseline to 43.79% in the modified model. Training and validation loss were also far higher than the baseline at every epoch, for example the final validation loss was 1.603 compared to 0.444 in the baseline. Since the backbone stays entirely frozen throughout this experiment, no backbone gradients are ever computed during training, so this drop cannot be explained as a gradient-flow problem in the usual sense (verified especially by the not so significant magnitude of the early gradient signal as compared for both model iterations). The real cause is a mismatch in the forward pass. The batch-normalization layers right after the three modified blocks still hold running statistics that were calibrated during the original ImageNet pretraining, when skip connections were present. Removing the skip connection changes the statistical distribution of the output of modified blocks, and the frozen batch-normalization layers downstream then apply their now-mismatched normalization to this shifted distribution. This corrupts the resulting feature and the distortion keeps compounding through every layer after the modification. This experiment is therefore really testing how well ImageNet-pretrained weights hold up under an architectural change they were never trained for, rather than testing gradient vanishing during training from scratch. Convergence was also clearly slower and worse overall. At epoch 5, the modified model's training loss (1.597) was

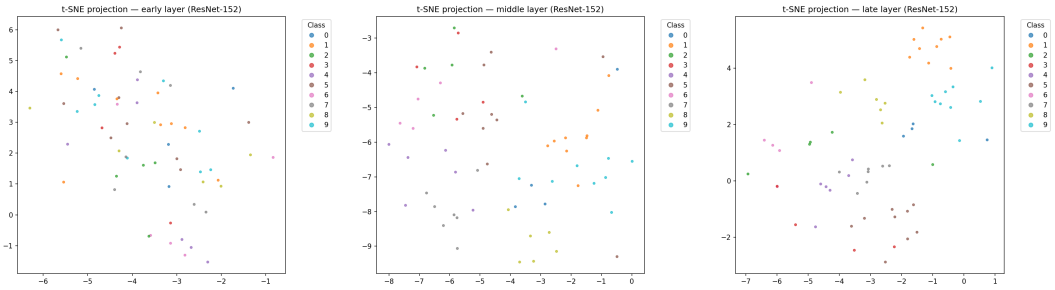
still far above even the baseline's epoch-1 loss (0.718), and validation accuracy stayed around 44 to 45% for the whole run instead of approaching the baseline's 83 to 85% range at any point.

1.3 Feature Hierarchies and Representations

1.3.1 Implementation

Forward hooks were registered on the outputs of `layer1` (early), `layer3` (middle), and `layer4` (late) of the trained baseline model. A single batch of CIFAR-10 validation images was passed through the network. Each hooked layer's raw feature map was global-average-pooled to a single value per channel and flattened, yielding one feature vector per image per layer (dimensionality equal to that layer's channel count: 256, 1024, and 2048 respectively). Each of the three resulting feature sets was independently reduced to 2 dimensions via t-SNE (PCA initialization) and plotted, colored by true class label. PCA initialization was used instead of random initialization for the t-SNE plots, since it gives the optimization a sensible, data-distribution informed starting layout rather than an random one, which makes the resulting plots more stable and reproducible across runs.

1.3.2 Findings and Discussion



(a) Early (layer1) (b) Middle (layer3) (c) Late (layer4)

Figure 3: t-SNE projections of pooled feature representations, colored by true class, at three network depths.

The three t-SNE projections show a clear progression in class separability with network depth. At the early layer, points from different classes are mixed with no visible structure. For example, class 6 and class 2 points sit directly adjacent to points from several unrelated classes, and no class forms even a loose grouping. At the middle layer, partial structure begins to emerge: classes such as 4 and 7 form loose, though still overlapping, regional groupings, and class 9 begins to cluster toward the upper right of the plot, though several classes remain visually intermixed. By the late layer, class separability is clearly the strongest of the three: class 1 forms a tight, distinct cluster at the top of the plot, class 9 forms its own separate block immediately below it, and classes 4, 7 and 8 each occupy their own recognisable regions, with comparatively little overlap between them.

This progression matches the expected behaviour of hierarchical feature learning in a CNN. Early layers are known to encode low-level, less class diffrentiating visual patterns, such as edges, colour gradients and basic textures, which are shared across many unrelated object categories. Since the raw pixel-level differences between an airplane and a ship may involve similar edge and colour statistics, low-level features alone give the network little basis to separate them, which is exactly what the near-random early-layer plot shows. Asdepth increases, the network combines these primitive patterns into increasingly complex, semantically meaningful combinations that are specific to particular object categories, which produces the clearer, more class-discriminative clustering seen at the late layer. This directly explains why a linear classifier trained only on frozen late-layer features (baseline model, subtask 1) is able to reach over 85% test accuracy: by the time features reach that depth, images of the same class are already positioned close together in feature space, so a comparatively simple linear boundary is sufficient to separate them.

1.4 Transfer Learning and Generalization

1.4.1 Implementation

Four model variants were trained, crossing initialization (ImageNet-pretrained vs.) with trainable depth (final block layer-4 + head only, vs. full backbone + head):
(i) Run A (pretrained, final block)
(ii) Run B (pretrained, full backbone)
(iii) Run C (random init, full backbone)
(iv) Run D (random init, final block)
All four were trained for 3 epochs with Adam (lr = 0.0001) on the identical CIFAR-10 train to val to test split used throughout Task 1.

1.4.2 Findings and Discussion

Table 3: Transfer learning comparison: final test accuracy and trainable parameters.

Run	Init.	Trainable scope	Params	Test Acc.
A	Pretrained	Final block + head	14,985,226	90.70%
B	Pretrained	Full backbone	58,164,298	96.63%
C	Random	Full backbone	58,164,298	54.40%
D	Random	Final block + head	14,985,226	38.04%

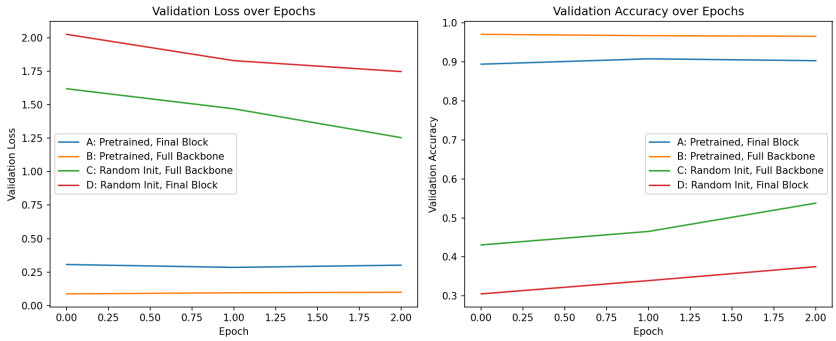


Figure 4: Validation loss and accuracy over training, all four transfer-learning runs.

The four transfer-learning runs isolate the effects of initialization and trainable depth. Comparing Run B (pretrained, full backbone, 96.63% test accuracy) against Run C (random initialization, full backbone, 54.40%) holds trainable depth and parameter count exactly fixed and varies only the starting weights. The resulting gap demonstrates that ImageNet pretraining provides a significant advantage even when the entire network is later allowed to fine-tune.

Comparing Run A (pretrained, final block only, 90.70%) against Run B (pretrained,full backbone, 96.63%) holds initialization fixed and varies only the number of trainable parameters. Unlocking the full backbone for training improves accuracy by approximately 6 percent over adapting only the final block, and both comfortably exceed the frozen-backbone baseline from subtask 1 (85.26%). This confirms the pattern noted in that subtask: a linear probe on entirely frozen features has a ceiling, and allowing progressively more of the network to adapt surpasses that ceiling, with the largest single gain coming from unfreezing at all (baseline to Run A, 5.4 percent increase) and a further, smaller gain from unfreezing the entire backbone (Run A to Run B, 5.9 percent increase).

Run D (random initialization, final block only, 38.04%) performed worst of all four configurations, which is expected: with 149 of ResNet-152's 152 layers permanently frozen at random initialization, the network has almost no capacity to extract meaningful features at all, and the small trainable portion cannot compensate.

In terms of compute-accuracy trade-off, Run A reaches 90.70% test accuracy while training only 14.9 million parameters, compared to Run B's 96.63% at 58.2 million trainable parameters, roughly four

times as many. If compute or training time is constrained, Run A offers a strong result at a lower of compute cost. If maximum accuracy is the priority and compute allows it, full-backbone fine-tuning from pretrained weights (Run B) is the best choice.

2 Task 2: Vision Transformer

2.1 Classification and Attention Visualization

2.1.1 Implementation

A pretrained `google/vit-base-patch16-224` model was used for all parts of this task. Two test images were selected: a photo of two cats on a couch, and a photo of a dog, each resized to 224×224 and split into a 14×14 grid of 16×16 patches.

For classification and attention visualization, the model's attention weights were extracted using `output_attentions=True`, and the final transformer layer's CLS-token attention row (its attention to each of the 196 patch tokens) was averaged across all 12 attention heads, reshaped to 14×14, and increases to the original image resolution to produce an overlay.

For the patch-masking experiment, patches were set to zero before being passed through the model, using two masking strategies: random masking, where a random subset of the 196 patches is zeroed out, and center masking, where the patches in the middle of the image are zeroed out instead. This was tested at two masking ratios, 25% and 50%, on both test images, and it was checked whether the model's prediction changed compared to the original, unmasked prediction.

For the CLS-versus-mean-pooling comparison, two separate linear classification heads were trained on top of the frozen ViT backbone, using a small subset of CIFAR-10(2000 training images, 500 test images) for speed. One head used the CLS token's final vector as input, and the other used the mean of all 196 patch tokens' final vectors instead. Both heads were trained for 5 epochs with the Adam optimizer, and their test accuracies were compared.

2.1.2 Findings and Discussion

The CLS-token attention overlays, averaged across all heads in the final transformer layer, show mixed but decipherable behaviour across the two test images. For `cats_on_couch` (predicted "Egyptian cat" with 93.7% confidence), attention correctly concentrates on both cats, with the strongest hotspot on the left cat's head and shoulder region and a secondary strong region on the right cat's head, closely matching the predicted class. However, a clearly irrelevant hotspot also appears on an empty region of pink couch fabric at the bottom of the frame, with no corresponding object content. For the `dog` image (predicted "Samoyed" with a notably lower 69.6% confidence), attention on the dog itself is comparatively weak and diffuse, while the strongest attention hotspots fall on background foliage and grass rather than the dog. This is a clear instance of the model attending substantially to irrelevant background regions.

It is worth noting that this overlay only shows the attention pattern from the final transformer layer, while the CLS token's final vector, which is what actually feeds the classifier, is shaped by attention accumulated across all 12 layers. It is possible that earlier layers focused more directly on the dog itself, and the final layer's broader attention reflects a later refinement step rather than the whole decision-making process. The lower confidence for the dog prediction may also partly reflect genuine visual similarity between Samoyed and other white, fluffy dog breeds in ImageNet's class set, rather than attention failing outright, since the prediction was still correct. Fully checking this would require visualizing attention at several different layers, not just the last one, which was outside the scope of this analysis.

Conceptually, this attention-based explanation differs from CNN interpretation methods such as CAM or Grad-CAM in an important way. For a CNN, figuring out which part of the image mattered for a prediction requires an extra calculation done after the prediction is already made, typically using gradients from a chosen convolutional layer to estimate which spatial regions were most important. A ViT's attention weights, by contrast, are numbers the model already calculated and used while making the prediction itself, since the CLS-token attention row used for the overlay above is literally the same weighting the model used internally to decide how much information to gather from each patch. No extra calculation after the fact is needed. This analysis used the head-averaged attention

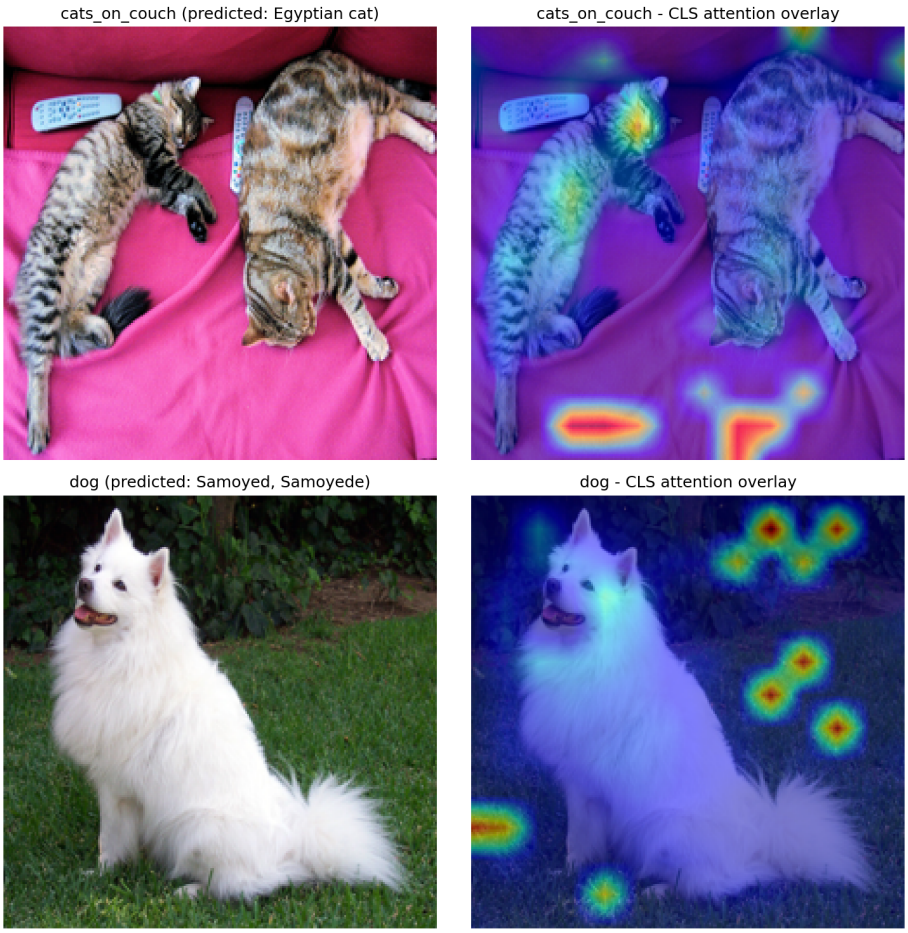


Figure 5: CLS-token attention overlays (final layer, averaged across heads) for two test images. Left: original image with predicted class. Right: attention overlay.

map rather than visualizing individual heads separately, so whether different heads are specialized cannot be directly determined from these results. This could be checked further by extracting and plotting each of the 12 heads' attention maps separately, rather than averaging them together, and seeing whether different heads consistently focus on different kinds of regions across several images.

Table 4: Prediction retained after patch masking, out of 2 test images.

Masking mode	25% masked	50% masked
Random	2/2	2/2
Center	1/2	1/2

The model was fully robust to random masking, keeping the correct prediction for both test images even with half of the patches removed. This makes sense because random masking removes patches scattered all over the image, so enough of the object usually stays visible somewhere for the model to still recognize it, and the remaining patches can still attend to each other to fill in a reasonable picture of the object. Center masking was much more damaging, since it removed patches from the middle of the image where the main object sits, and it caused the prediction to change for one of the two images at both masking ratios. This shows that the model is not just relying on a few scattered informative patches, it specifically relies on the central region where the object usually is, and removing that region specifically is much more damaging than removing the same number of patches picked at random.

Table 5: Linear probe accuracy, CLS token vs. mean-pooled patch tokens.

Pooling method	Test Accuracy
CLS token	96.20%
Mean-pooled patches	96.60%

The mean-pooled probe slightly outperformed the CLS probe, 96.60% versus 96.20%, though the difference is small. This is somewhat different from what might be expected, since the CLS token is specifically trained to gather a useful, weighted summary of the image, while mean-pooling just averages every patch equally with no learned weighting at all. One possible explanation is that CIFAR-10 images are fairly simple and mostly object-centered, so a plain average of all patches already captures most of the useful information, without needing the extra flexibility that CLS's learned weighting provides. Another possible factor is that the CLS token was originally trained together with the rest of the pretrained model for ImageNet classification specifically, so its learned weighting may be tuned toward what was useful there rather than being a generally superior summary for any downstream task. This suggests that the choice of pooling method can interact with the pretraining objective. Since this ViT checkpoint was pretrained with a classification goal using the CLS token, it might favor CLS, but the small gap here suggests that for a comparatively simple task like CIFAR-10 classification, a plain average of patch tokens can be just as effective as a summary CLS vector from a frozen backbone.

3 Task 3: CLIP

3.1 Zero-Shot Classification and the Modality Gap

3.1.1 Implementation

The pretrained CLIP model (ViT-B/32 image encoder) was used and tested on the full STL-10 test set (8000 images) without any training, since CLIP can do zero-shot classification. Three different ways of writing the text prompts were tried: just the plain class name, a template like "a photo of a {class}", and a longer, more descriptive prompt. For the second part, 100 images from STL-10 were taken along with their correct class-name captions and CLIP embeddings were obtained for both the images and the text. The distance between the average image embedding and the average text embedding was measured to see how far apart the two are. Then an orthogonal Procrustes alignment was applied (finding a rotation matrix that lines up the image embeddings with the text embeddings as closely as possible), and it was checked whether this changed the gap and whether it changed the accuracy.

3.1.2 Findings and Discussion

Table 6: Zero-shot classification accuracy on STL-10 (full test set) by prompting strategy.

Prompting strategy	Accuracy
Plain label	96.26%
"a photo of a {class}"	97.36%
Descriptive	95.74%

Out of the three prompt styles, "a photo of a {class}" worked best at 97.36% accuracy, plain labels got 96.26%, and the longer descriptive prompt actually did a bit worse at 95.74%. This makes sense because CLIP was trained on real image captions from the internet, and people don't usually just write a single word like "cat" as a caption, they write full sentences. So a prompt that looks more like a real caption works better. The longer prompt did not help, perhaps because it added extra words that weren't really matching what CLIP saw during training.

The modality gap becomes apparent when plotted. Before alignment, the image points and text points form two separate groups in the t-SNE plot. The distance between the average image point and average text point was 1.044. This is surprising because these are the correct matching pairs, so it might be expected that they would be close together.

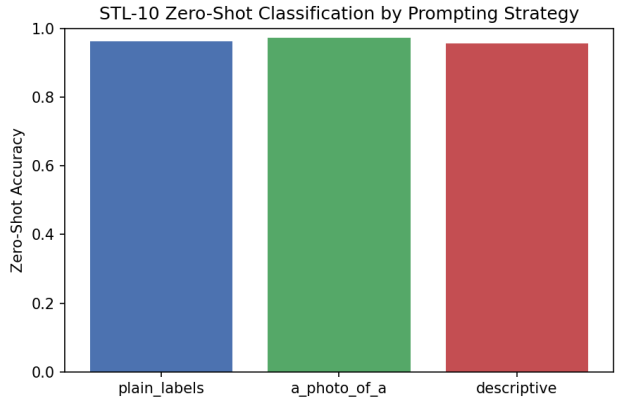
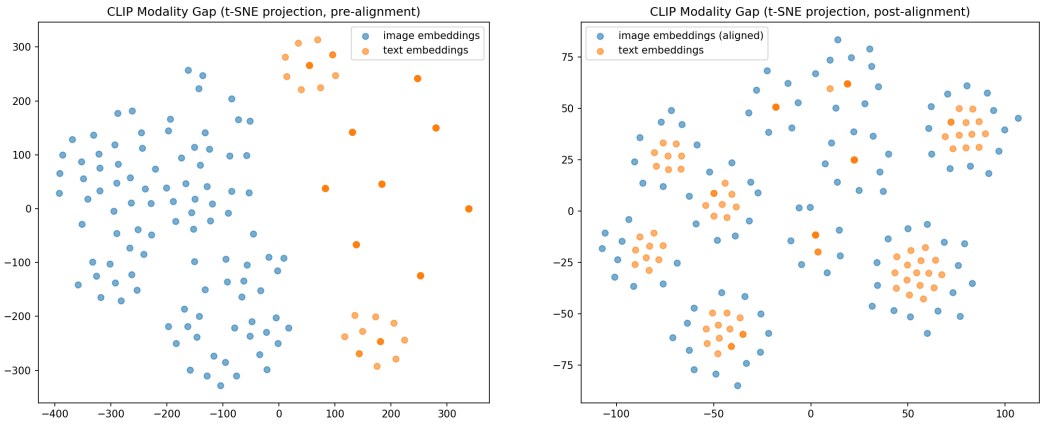


Figure 6: Zero-shot classification accuracy by prompting strategy, STL-10 test set.



(a) Before Procrustes alignment

(b) After Procrustes alignment

Figure 7: t-SNE projection of image and text embeddings before and after orthogonal Procrustes alignment (100 sampled STL-10 image–caption pairs).

After the Procrustes alignment was applied, the gap dropped by 87%. The plot after alignment shows the image and text points now mixing together in the same small clusters instead of being two separate sections. This suggests the gap is mostly just a simple rotation/offset between the two encoders, not some totally different structure. Although the gap shrunk , the accuracy did not change by much , 97.36% before and 97.14% after. This is because zero-shot classification only cares about which text prompt is closest to the image compared to the other prompts, not whether the image and text are close in absolute terms. So even with the gap present, the relative distances were already fine for classification, which is why fixing the gap did not marginally improve accuracy.

4 Task 4: Variational Autoencoder

4.1 VAE on MNIST

4.1.1 Implementation

An MLP-based VAE was built for MNIST. The encoder has one hidden layer (784 to 400, ReLU), followed by two separate linear layers that produce the mean and log-variance of a 2-dimensional latent Gaussian distribution (400 to 2 for each). The latent dimension was kept at 2 so the latent space could be plotted directly without needing extra dimensionality reduction. The reparameterization trick was used to sample $z = \mu + \sigma \odot \epsilon$, with $\epsilon \sim \mathcal{N}(0, I)$. The decoder mirrors the encoder (2 -> 400 -> 784) and outputs pixel logits, matched with a Bernoulli output distribution. The loss was

the negative ELBO: binary cross-entropy reconstruction loss plus the closed-form KL divergence between $\mathcal{N}(\mu, \sigma^2)$ and $\mathcal{N}(0, I)$. The model was trained for 10 epochs with the Adam optimizer (lr $= 0.001$) and a batch size of 128.

4.1.2 Findings and Discussion

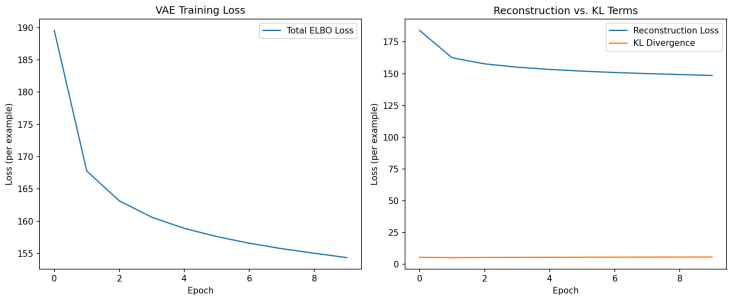


Figure 8: Total ELBO loss and its reconstruction/KL components over training.

The total ELBO loss dropped from 189.6 to 154.4 over 10 epochs and was still decreasing at the end of training rather than fully flattening out, so more epochs would likely have improved results. The reconstruction loss makes up almost all of the total loss (148.6 out of 154.4 at the final epoch), while the KL term stays small and roughly flat (around 5.5 to 5.75 throughout training). This is expected for a small 2-dimensional latent space, since there is not much room for the encoder's distributions to shift far from the prior.

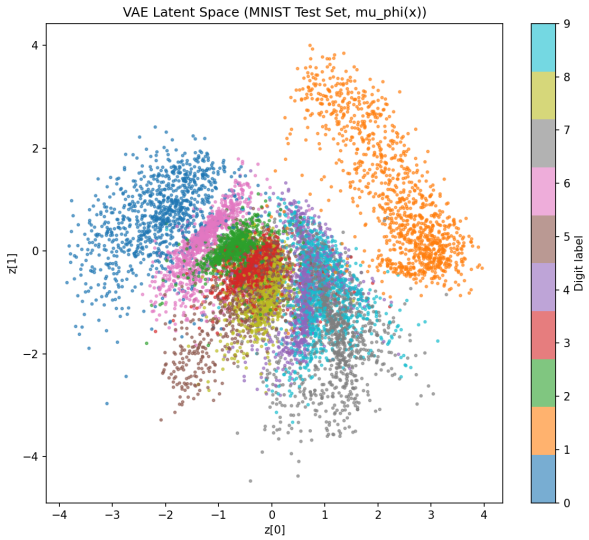


Figure 9: Latent space ($\mu_\phi(x)$) of the MNIST test set, colored by digit label.

The latent space plot shows that some digits are organized clearly and others are not. Digits 0 (blue) and 1 (orange) form their own clearly separated regions, in the top-left and top-right of the plot. Most of the other digits (2 through 9) end up crowded together in one large central region with a lot of overlap between classes, though there is some rough structure, for example 7 tends to sit more toward the lower-right and 5 more toward the bottom. This suggests the model learned to separate the digits that look most visually distinct (0 and 1 have very different shapes from everything else) much more easily than the digits that share more visual similarity with each other.

The reconstructions are blurry, which is a well-known property of VAEs since the reconstruction loss effectively averages over the range of plausible outputs rather than committing to sharp pixel values. Digits 0 and 1 are reconstructed clearly and correctly. However, several other digits are reconstructed

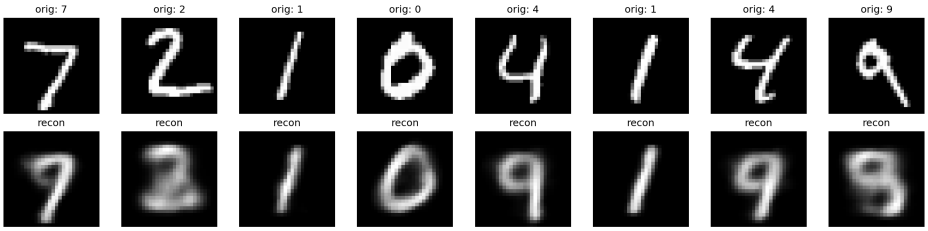


Figure 10: Original MNIST test images (top row) and their VAE reconstructions (bottom row).

incorrectly: for example, the original 7 is reconstructed as something closer to a 9, and two separate images of 4 are both reconstructed as shapes closer to 9 as well. This matches what the latent space plot shows: since 4, 7, and 9 are not cleanly separated in latent space and sit close to each other in the crowded central region, their reconstructions get mixed.

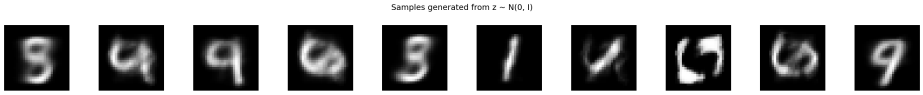


Figure 11: Samples generated by decoding random $z \sim \mathcal{N}(0, I)$.

Most of the generated samples look like plausible handwritten digits. Several are clearly readable (5, 9, 1), which shows the decoder learned a reasonable mapping from the prior back to image space. A few samples are more ambiguous and do not clearly match any single digit, which is consistent with the latent space plot, since a random point sampled from $\mathcal{N}(0, I)$ has a good chance of ending up in the crowded central region where several different digit classes overlap, producing a less distinct shape.

Compared to Doersch's tutorial implementation, this VAE uses the same overall approach: a simple MLP encoder and decoder, a Bernoulli output distribution for the pixels, and binary cross-entropy (via `binary_cross_entropy_with_logits`) as the reconstruction loss, matching Doersch's use of sigmoid cross-entropy for MNIST. The main difference is the latent dimension. This implementation deliberately used d = 2 so the latent space could be visualized directly, while Doersch's default examples typically use a higher-dimensional latent space. Based on the results: a 2-dimensional latent space is easy to plot and interpret, but it appears small to cleanly separate all ten digit classes, which likely explains some of the reconstruction confusion observed between visually similar digits such as 4, 7, and 9. A higher-dimensional latent space would probably reduce this class overlap and improve reconstruction accuracy, at the cost of no longer being directly plottable without an additional dimensionality reduction step such as t-SNE.